Open Astro Core

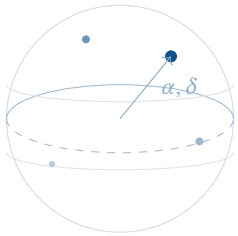
A Pure-Function SDK for Computational Astrophotography

Simon Knight

Adelaide, Australia

March 2026 • Version 0.4.0

Last updated: March 11, 2026



Abstract. Astrophotography demands a wide range of computational primitives—coordinate transforms, celestial mechanics, image statistics, noise modelling, frame registration, and stacking—yet existing tools scatter this logic across monolithic GUI applications with tightly coupled I/O. Open Astro Core (`astro-core`, `astro-vision`) is a Rust SDK that provides these primitives as pure, deterministic functions with no I/O or threading dependencies. This report describes the layered architecture, formalises the key algorithms (coordinate transforms, CMOS noise modelling, sigma-clipped stacking, triangle-similarity registration, and WCS with SIP distortion), and discusses the design rationale that enables the SDK to be embedded in contexts ranging from a Raspberry Pi edge daemon to an iOS framing assistant. At the time of writing the codebase comprises 54,130 lines of Rust across 130 source files with 2,198 unit tests.

Version	Date	Changes
0.4.0	March 2026	Architectural & Edge Optimization: native plate solver, mobile bridge, power management, AR math.
0.3.0	March 2026	WCS/SIP, mosaic planning, autofocus, co-axial guiding.
0.2.0	March 2026	Imaging intelligence: stats, stacking, exposure, FITS.
0.1.0	Feb 2026	Initial release; celestial math foundation.

Contents

List of Design Decisions & Rules

1 Introduction

Modern astrophotography is a computationally intensive discipline. A single deep-sky imaging session may require the operator to compute target visibility windows from celestial mechanics, derive optimal sub-exposure times from a camera noise model, calibrate raw frames against darks and flats, register sub-exposures by matching star patterns, and stack hundreds of frames with outlier-robust statistics.

Traditional tools—PixInsight, Siril, NINA—provide these capabilities, but their implementations are embedded inside monolithic applications with tightly coupled user interfaces and file I/O. This coupling makes it difficult to reuse individual algorithms in new contexts: an iOS framing assistant, a Raspberry Pi edge daemon, or a command-line automation pipeline.

Insight: *The astrophotography pipeline mirrors signal processing: acquire, calibrate, align, integrate, enhance. Each stage has well-defined mathematical foundations that benefit from isolation.*

Open Astro Core addresses this gap by providing a layered Rust Software Development Kit (SDK) in which every algorithm is a *pure function*: it takes typed inputs, produces typed outputs, and performs no I/O, no heap allocation beyond its return value, and no threading. This design yields three practical benefits:

1. **Embeddability.** The SDK compiles to a static library with no runtime dependencies, making it trivially embeddable via Foreign Function Interface (FFI) in Swift (iOS), Node.js (Electron), or Python (PyO3).
2. **Testability.** Every function is deterministic and can be tested with a simple assertion; no mocking of hardware, filesystems, or network connections is required.
3. **Composability.** Higher-level systems (e.g. a TUI orchestrator, an Instrument Neutral Distributed Interface (INDI) driver, or an HTTP API) compose SDK functions without inheriting implicit state or side effects.

This report makes the following contributions:

1. A description of the layered crate architecture and its dependency hierarchy (section 3).
2. A formalisation of the core coordinate transform pipeline, including sidereal time, equatorial-to-horizontal conversion, and atmospheric refraction (section 4).
3. A presentation of the imaging algorithms: Complementary Metal-Oxide-Semiconductor (CMOS) noise modelling, sigma-clipped stacking, triangle-similarity registration, and background estimation (section 5).
4. A description of the World Coordinate System (WCS) implementation with full Simple Imaging Polynomial (SIP) distortion polynomial support (section 6).
5. A discussion of the design rationale behind the pure-function constraint and its consequences for error handling and composability (section 7).

2 Background

Astrophotography processing involves a well-established pipeline: acquire, calibrate, register, stack, and stretch. The mathematical foundations draw from three domains.

2.1 Positional Astronomy

Positional astronomy provides the spherical trigonometry needed to convert between equatorial coordinates (right ascension, declination) and the observer's local horizon system (altitude, azimuth). The standard references are Meeus [1] for practical algorithms and the IAU SOFA library [2] for high-precision implementations. Open Astro Core follows the Meeus approach for its balance of accuracy and simplicity.

2.2 CMOS Noise Theory

The Signal-to-Noise Ratio (SNR) of a CMOS exposure is governed by the photon statistics of the signal and sky background, the sensor's read noise, and the dark current. The standard imaging sensor noise equation [3, 4] is:

$$\mathrm{SNR} = \frac{S \cdot t}{\sqrt{S \cdot t + B_{\mathrm{sky}} \cdot t + D \cdot t + N \cdot \sigma_R^2}} \tag{1}$$

where S is the signal rate (e^-/s), t is the exposure time, B_{sky} is the sky background rate, D is the dark current, N is the number of stacked sub-exposures, and σ_R is the read noise in electrons. This equation underpins the exposure recommendation engine described in section 5.6.

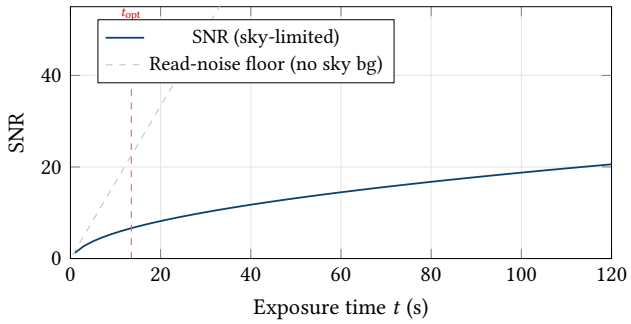


Figure 1. SNR as a function of sub-exposure time for a typical CMOS sensor. The dashed red line marks t_{opt}, the "swamp the read noise" threshold. Above this point the sky background noise dominates read noise and further extending the sub-exposure yields diminishing returns; the same total SNR can be achieved by stacking more shorter exposures.

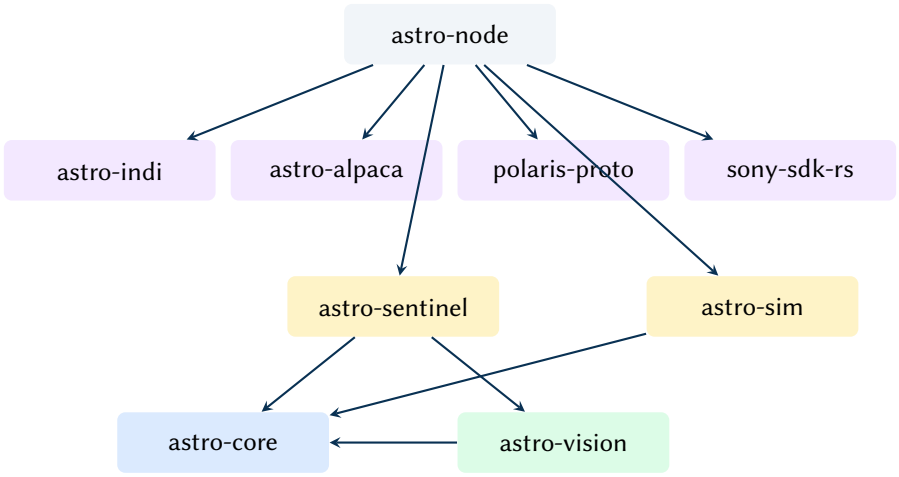


Figure 2. Crate dependency hierarchy. Arrows point from dependant to dependency. The two crates at the base contain all pure computation; upper layers add I/O and device interaction.

2.3 Image Registration

Sub-exposure registration requires identifying corresponding stars between frames and computing a geometric transform that aligns them. Triangle-similarity matching [5, 6] is the dominant approach in astronomical software: triangles formed by bright stars are described by their sorted side ratios, which are invariant to translation, rotation, and uniform scaling. Open Astro Core implements this algorithm with a RANSAC-style [7] consensus step, described in section 5.7.

3 Architecture

Open Astro Core is organised as a Cargo workspace containing eleven crates. The crates form a strict dependency hierarchy: lower layers provide pure computation; upper layers add I/O, hardware interaction, and orchestration.

3.1 Crate Hierarchy

Figure 2 illustrates the dependency graph. The two foundation crates—`astro-core` and `astro-vision`—contain all algorithms discussed in this report. They have no dependencies on tokio, device drivers, or file I/O libraries beyond the Flexible Image Transport System (FITS) and RAW format parsers in `astro-vision`.

Insight: *The "inverted pyramid" ensures that the most-reused code (math primitives) has zero dependencies beyond `serde` and `thiserror`.*

: Dependency Inversion

Foundation crates (`astro-core`, `astro-vision`) must never depend on I/O, async runtimes, or device-specific code. All hardware interaction is mediated through traits defined in the foundation and implemented in upper-layer crates.

3.2 Foundation: astro-core

The foundation crate providing coordinate math, celestial mechanics, and planning primitives. Key modules include:

- `angle`, `sexagesimal` — angle newtypes with wrapping arithmetic and HMS/DMS parsing.
- `transforms` — equatorial↔horizontal conversion, angular separation (haversine), atmospheric refraction (Bennett's formula).
- `time` — Julian Date, Greenwich Mean Sidereal Time (GMST), Local Sidereal Time (LST) computation.
- `visibility` — airmass (Pickering), rise/transit/set, altitude curves.
- `solar`, `lunar` — Meeus-derived solar and lunar ephemeris with eclipse prediction.
- `session` — imaging window scoring, parallactic angle, field rotation rate, max untrailed exposure.
- `wcs` — WCS with TAN projection and SIP distortion polynomials (section 6).
- `mosaic` — mosaic grid generation with RA wrapping, serpentine ordering, and coverage tracking.
- `guiding` — co-axial offset calibration, guide quality analysis (RMS, periodic error FFT), spiral/grid search patterns.

3.3 Foundation: astro-vision

Image processing algorithms operating on `Array2<f32>` from the `ndarray` crate. All pixel data is normalised to the $[0, 1]$ range.

- `stats` — median, MAD, sigma-clipped statistics, histogram, centroid.
- `background` — tiled sigma-clipped background estimation with bilinear interpolation.
- `stardetect` — threshold-and-flood-fill star detection with Half-Flux Radius (HFR) and Full Width at Half Maximum (FWHM) measurement.
- `stacking` — mean, median, sigma-clipped mean, and Winsorized sigma clipping.
- `registration` — triangle-similarity star matching with RANSAC affine fitting.
- `calibrate` — dark/flat/bias calibration with hot/cold pixel detection and interpolation.
- `noise_model` — CMOS noise model with SNR computation and optimal sub-exposure derivation.
- `exposure` — exposure recommendation, saturation prediction, Holy Grail ramp for timelapse.
- `autofocus` — V-curve fitting with IRLS Huber weights.
- `fits_write`, `ser` — FITS and Simple Extended Recording (SER) file I/O.
- `stretch` — Midtone Transfer Function (MTF) auto-stretch.

3.4 Mid-Tier Crates

3.4.1 astro-sentinel

A rules engine that evaluates image classification results against TOML-defined rule sets. Optionally backed by an ONNX inference runtime for edge-AI classification (wildlife detection, cloud detection). The pipeline produces structured `Explanation` objects with matched rules and confidence scores.

3.4.2 astro-sim

A Hardware-in-the-Loop (HITL) virtual rig with mount kinematics (slewing, periodic error injection, German Equatorial Mount (GEM) meridian flip), multi-device axis offsets, Gaussian PSF star rendering, and target fitness scoring. The simulator enables full-pipeline testing without physical hardware.

3.5 Driver Crates

`astro-indi` and `astro-alpaca` implement the two dominant telescope control protocols (INDI and Astronomy Common Object Model (ASCOM) Alpaca). `polaris-proto` is a native binary protocol driver for the Benro Polaris star tracker. `sony-sdk-rs` provides safe Rust bindings over the Sony Camera Remote SDK via a C ABI wrapper (`sony-sdk-sys`).

4 Celestial Mechanics

This section describes the core positional astronomy pipeline: sidereal time, coordinate transforms, and atmospheric refraction.

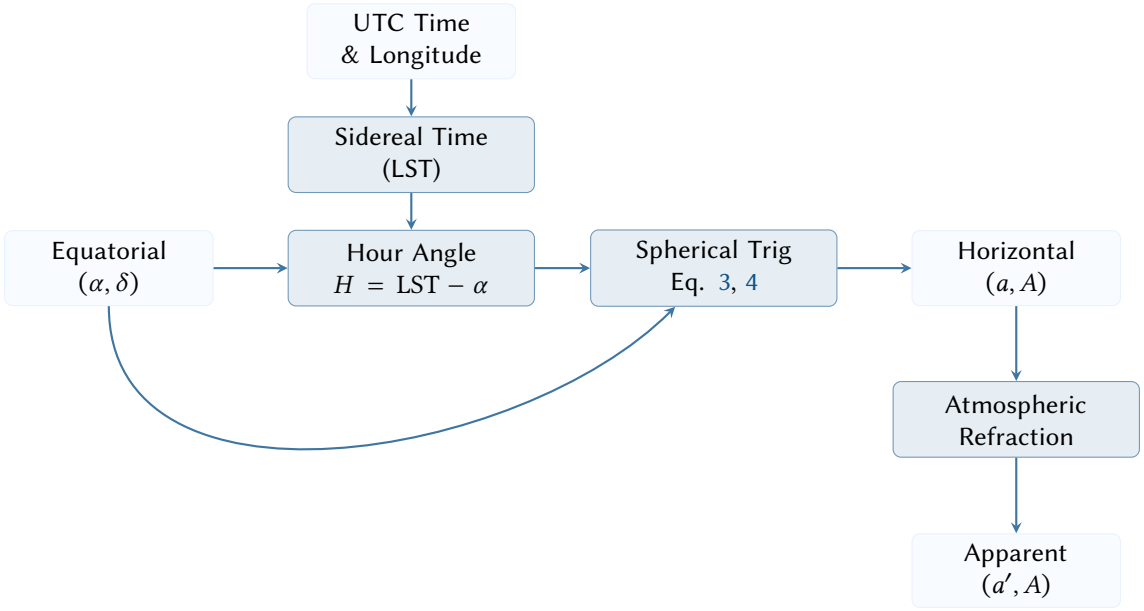


Figure 3. Coordinate transform pipeline mapping equatorial targets to their apparent topocentric positions.

4.1 Sidereal Time

The GMST converts a Julian Date JD to the hour angle of the vernal equinox. Open Astro Core uses the Meeus cubic approximation [1]:

$$\Theta_{\text{GMST}} = 280.46061837 + 360.98564736629\,(JD - 2451545.0) + 0.000388\,T^2 - \frac{T^3}{38710000} \tag{2}$$

where $T = (JD - 2451545.0)/36525$ is the Julian century. LST is then LST = GMST + $\lambda/15$ where λ is the observer's east longitude in degrees.

> **Design Decision 1: Why Meeus Over SOFA?**
>
> The IAU SOFA library provides sub-milliarcsecond precision through nutation and precession corrections. For astrophotography—where the dominant error sources are polar alignment (~1 arcmin), atmospheric refraction (~30 arcsec), and mount periodic error (~10 arcsec)—the Meeus approximation's ~1 arcsec accuracy is more than sufficient, and the resulting code is self-contained with no external C dependencies.

4.2 Equatorial-to-Horizontal Transform

Given equatorial coordinates (H, δ) where H is the hour angle and δ is the declination, the standard spherical trigonometry yields the altitude a and azimuth A:

$$\sin a = \sin\phi\,\sin\delta + \cos\phi\,\cos\delta\,\cos H \tag{3}$$

$$A = \operatorname{atan2}(-\cos\delta\,\sin H,\quad \sin\delta\,\cos\phi - \cos\delta\,\cos H\,\sin\phi) \tag{4}$$

where ϕ is the observer's latitude. The azimuth follows the ASCOM convention: North = 0, East-positive (clockwise).

The implementation clamps the $\sin a$ argument to $[-1, 1]$ to prevent `NaN` from floating-point rounding at the zenith and nadir.

Insight: The North=0 East-positive convention differs from the mathematical standard (East=0 counter clockwise). Matching ASCOM avoids a conversion step in every driver.

Listing 1. Equatorial-to-horizontal transform.

```
pub fn equatorial_to_horizontal(
    eq: EquatorialCoord,
    lst: RightAscension,
    site_lat: Latitude,
```

```
) -> HorizontalCoord {
    let phi = site_lat.0.to_radians();
    let dec = eq.dec.0.to_radians();
    let h = Angle(lst.to_angle().0 - eq.ra.to_angle().0)
        .wrap_pi().0;

    let sin_alt = phi.sin() * dec.sin()
        + phi.cos() * dec.cos() * h.cos();
    let alt_rad = clamp_unit(sin_alt).asin();

    let y = -dec.cos() * h.sin();
    let x = dec.sin() * phi.cos()
        - dec.cos() * h.cos() * phi.sin();
    let az_rad = Angle(y.atan2(x)).wrap_2pi().0;

    HorizontalCoord {
        az: Azimuth(az_rad.to_degrees()).normalized(),
        alt: Altitude(alt_rad.to_degrees()),
    }
}
```

4.3 Angular Separation

The angular distance between two points on the celestial sphere uses the haversine formula for numerical stability at small separations:

$$d = 2 \arcsin\sqrt{\sin^2\frac{\Delta\delta}{2} + \cos\delta_1 \cos\delta_2 \sin^2\frac{\Delta\alpha}{2}} \tag{5}$$

The naive dot-product formula $d = \arccos(\mathbf{u} \cdot \mathbf{v})$ loses precision below ~0.1 arcsec due to the flat gradient of arccos near unity. Since astrophotography routinely measures separations of a few arcseconds (e.g. guide star offsets, plate-solve residuals), the haversine form is essential.

4.4 Atmospheric Refraction

The apparent altitude of a celestial object is increased by atmospheric refraction. Open Astro Core applies Bennett's formula [8]:

$$R = \frac{1}{\tan\left(h + \frac{7.31}{h+4.4}\right)} \quad \text{(arcminutes)} \tag{6}$$

with a multiplicative temperature/pressure correction factor. This correction is applied in the rise/transit/set calculator, where it shifts the effective horizon from 0° to approximately $0^\circ 34'$.

4.5 Airmass

The airmass X quantifies the path length of light through the atmosphere relative to the zenith. Open Astro Core uses Pickering's improved formula [9], which remains accurate down to the horizon:

$$X = \frac{1}{\sin\left(h + \frac{244}{165+47\,h^{1.1}}\right)} \tag{7}$$

where h is the true altitude in degrees. The standard $\sec z$ approximation diverges below ~10° altitude; Pickering's formula agrees with rigorous ray-tracing models [10] to within 0.1% at all altitudes above 1°.

4.6 Lunar Ephemeris

The `lunar` module implements Meeus Chapter 47 with 10+ perturbation terms for ecliptic longitude and 6+ for latitude. The geocentric ecliptic position is converted to equatorial via the obliquity of the ecliptic, then corrected for topocentric parallax using the observer's geocentric latitude and distance.

Key derived quantities include:

- **Phase illumination** from the elongation angle D, accounting for perturbation corrections.
- **Eclipse prediction** via umbral/penumbral shadow geometry, with binary search for maximum eclipse and contact-time detection by stepping.

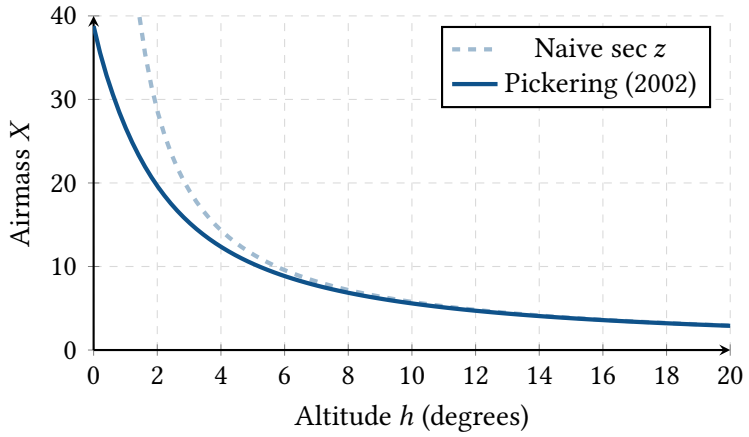


Figure 4. Comparison of airmass models near the horizon. The naive secant approximation diverges to infinity at $h = 0^\circ$, whereas Pickering's interpolation correctly models atmospheric curvature to match empirical scattering data ($X \approx 38$ at the horizon).

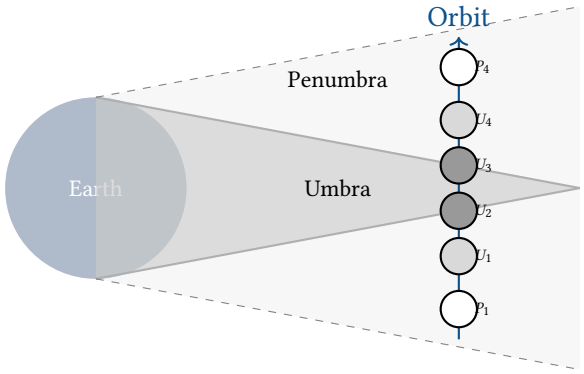


Figure 5. Lunar eclipse shadow geometry showing the Earth's umbral and penumbral cones. The SDK predicts contact points (P_1, U_1, U_2, etc.) by stepping through the Moon's geocentric ecliptic coordinates.

4.7 Polar Alignment Error & Drift

The `polar_align` module mathematically models the drift of a star caused by the misalignment between the mount's mechanical right ascension axis and the True Celestial Pole (TCP).

By tracking a star's apparent declination drift over time without guiding, the system solves the spherical trigonometry required to compute the exact azimuth and altitude offset vectors needed to physical align the mount, formalizing the traditional King Drift Method [11].

4.8 Target Visibility & Horizon Masks

The `horizon_mask` module models the local observing topology using a discrete altitude-azimuth polygon. Instead of assuming a flat 0° horizon, the planner intersects the celestial target's computed daily altitude curve with the local mask polygon to derive true unoccluded observing windows, preventing the scheduler from tracking targets behind trees or buildings.

4.9 Target Planning & Session Scoring

Choosing the right target for a given night requires combining multiple quality signals into a single actionable recommendation. The `session` module provides two complementary tools for this.

4.9.1 Twilight Boundaries

The `twilight_times` function bisection-searches the Sun's altitude curve to locate the six twilight boundaries—civil, nautical, and astronomical—for both dusk and dawn. This defines the usable imaging window for the night.

4.9.2 Imaging Window Score

The `imaging_window_score` function collapses four orthogonal quality signals into a single $[0, 1]$ score at a given Julian Date:

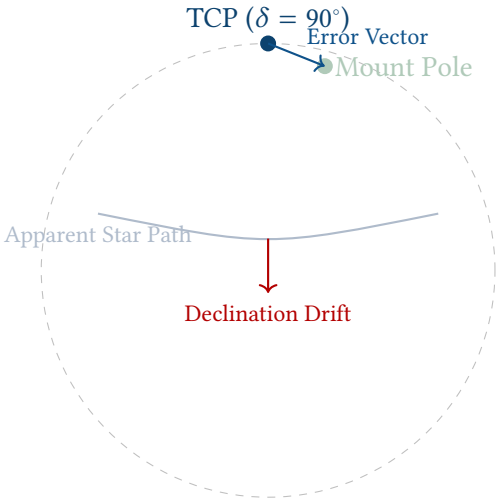


Figure 6. Polar alignment error. A misalignment between the True Celestial Pole (TCP) and the mount's mechanical polar axis causes stars to exhibit a continuous declination drift over time, which forms the basis of the King drift method.

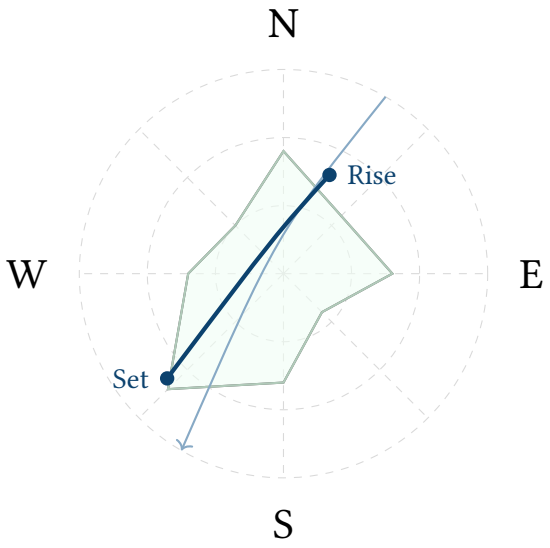


Figure 7. Altitude-Azimuth polar projection (zenith at center, horizon at edge). The shaded polygon represents the local topographical horizon mask (e.g., trees, buildings). The target's nightly arc intersects the mask to compute precise visibility windows.

$$Q = Q_{\text{alt}} \cdot Q_{\text{air}} \cdot Q_{\text{moon}} \cdot Q_{\text{dark}} \tag{8}$$

where:

- Q_{alt}: altitude factor—zero below horizon, linear from 0° to 30°, unity above 30°.
- Q_{air}: airmass factor—$1-(\mathcal{X}-1)/3$, clamped to $[0, 1]$. Penalises low-elevation targets whose light traverses more atmosphere.
- Q_{moon}: moon proximity factor—zero within 0°, unity beyond 60° angular separation, linear between.
- Q_{dark}: darkness factor—unity in astronomical darkness ($\odot < -18^\circ$), 0.5 in nautical twilight, zero otherwise.

Design Decision 2: Multiplicative Score Design

Multiplying four factors—rather than summing weighted components—ensures that a single disqualifying condition (below horizon, daytime, full Moon on target) collapses the score to zero regardless of how

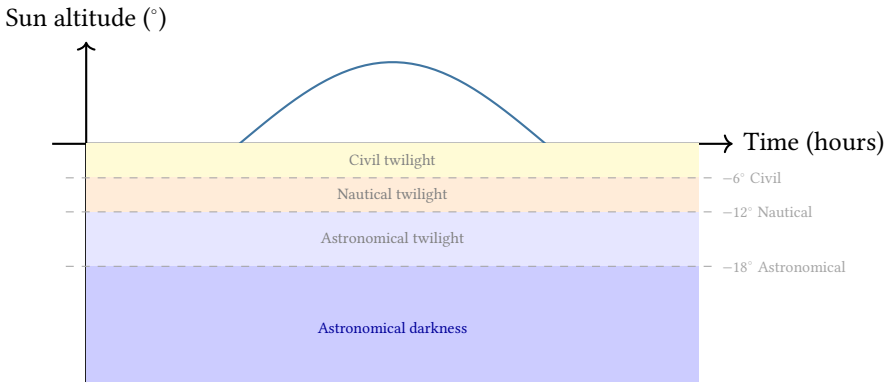


Figure 8. Twilight boundary model. The `twilight_times` function locates the six solar altitude crossings that delimit civil, nautical, and astronomical twilight, bounding the usable deep-sky imaging window.

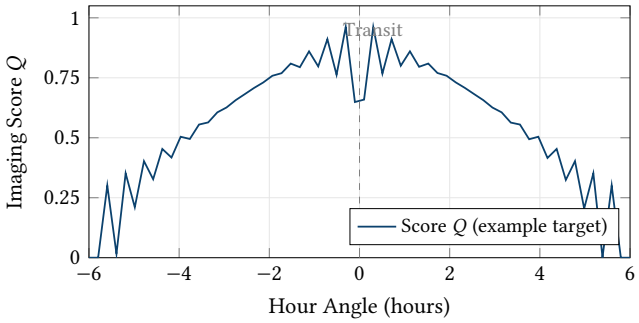


Figure 9. Imaging window score over a night for a representative target at declination +45°. The score peaks near transit (hour angle $H = 0$) where the target reaches maximum altitude and minimum airmass, then decays symmetrically as the target descends toward the horizon.

favourable the other signals are. This prevents the scheduler from selecting a technically high-altitude target that is flooded by moonlight.

4.10 Dome Slaving Geometry

For observatories housed within a rotating hemispherical dome, the dome slit must actively track the telescope's optical axis. However, the telescope's center of rotation is rarely identical to the geometric center of the dome sphere.

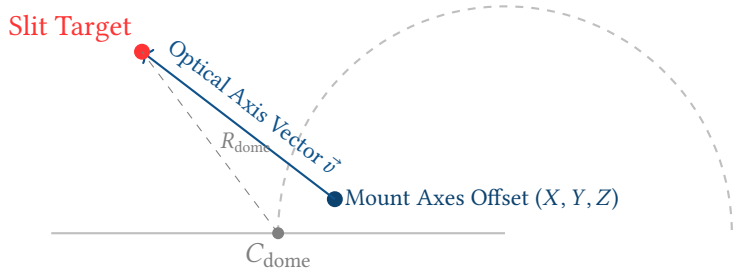


Figure 10. Dome slaving kinematics. Because the telescope mount is offset from the dome's geometric center, the optical axis $\vec{v}$ does not point to the same azimuth as the mount's pointing azimuth. The system must solve the ray-sphere intersection to command the correct dome rotation angle.

The `dome` module models this as a ray-sphere intersection problem [12]. Given the mount's (X, Y, Z) spatial offset from the dome center and the dome radius R, the system projects the optical vector $\vec{v}$ (derived from the current Altitude and Azimuth) to find the exact intersection point on the dome shell, commanding the slit to the corrected azimuth.

5 Imaging Algorithms

This section presents the core imaging algorithms in `astro-vision`: noise modelling, background estimation, frame stacking, and registration.

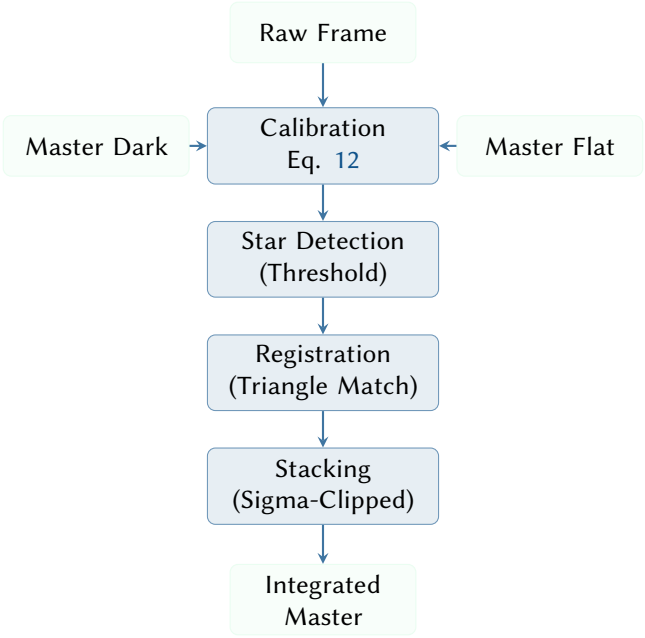


Figure 11. Image processing pipeline from raw sub-exposures to an integrated master frame.

5.1 CMOS Noise Model

The `noise_model` module encapsulates a camera's noise characteristics in a `CameraProfile` struct:

- `gain`: electrons per ADU (e^-/ADU)
- `read_noise`: RMS read noise in electrons
- `dark_current`: dark current in e^-/pixel/second
- `full_well`: full well capacity in electrons
- `bit_depth`: ADC resolution (typically 12 or 14 bits)

The SNR computation implements eq. (1) directly. When stacking N sub-exposures, the total signal scales as N while the uncorrelated noise components scale as $\sqrt{N}$, yielding a $\sqrt{N}$ improvement in SNR.

5.1.1 Optimal Sub-Exposure

The "swamp the read noise" strategy determines the minimum sub-exposure time t_{opt} at which the sky background dominates the read noise:

$$t_{\text{opt}} = \frac{(r \cdot \sigma_R)^2}{B_{\text{sky}}} \tag{9}$$

where r is a configurable ratio factor (default 3.0) and B_{sky} is the sky background rate in e^-/pixel/s. At this exposure time, the sky background noise is r times the read noise, ensuring that read noise contributes less than $1/(r^2+1) \approx 10\%$ of the total noise variance.

> **Design Decision 3: Swamp Ratio Default**
>
> The default ratio $r = 3.0$ ensures read noise is <10% of total variance. Higher ratios (e.g. $r = 5$) reduce the read noise contribution further but require proportionally longer sub-exposures, which may exceed the saturation limit at bright sites. The value 3.0 represents the community consensus for a practical compromise.

5.1.2 Sky Brightness Model

Sky background is derived from the Bortle scale via a magnitude-to-flux conversion. The nine-level `BortleScale` enum maps each class to a sky surface brightness in mag/arcsec^2 (ranging from 22.0 at Bortle 1 to 17.5 at Bortle 9):

Listing 2. Bortle scale sky brightness mapping.

```
pub fn surface_brightness(&self) -> f32 {
    match self {
        BortleScale::B1 => 22.0,
        BortleScale::B2 => 21.8,
        BortleScale::B3 => 21.6,
        BortleScale::B4 => 21.0,
        BortleScale::B5 => 20.0,
        BortleScale::B6 => 19.0,
        BortleScale::B7 => 18.5,
        BortleScale::B8 => 18.0,
        BortleScale::B9 => 17.5,
    }
}
```

The flux conversion accounts for the telescope aperture area, pixel scale, and sensor quantum efficiency. Built-in camera profiles are provided for the ZWO ASI294MC Pro, ASI533MC Pro, ASI2600MC Pro, and Canon 6D (modified).

5.1.3 Spectral Response & Narrowband Physics

The `CameraProfile` also characterizes the sensor's spectral Quantum Efficiency (QE), crucial for narrowband imaging (e.g., Hydrogen-alpha, Oxygen-III, Sulfur-II).

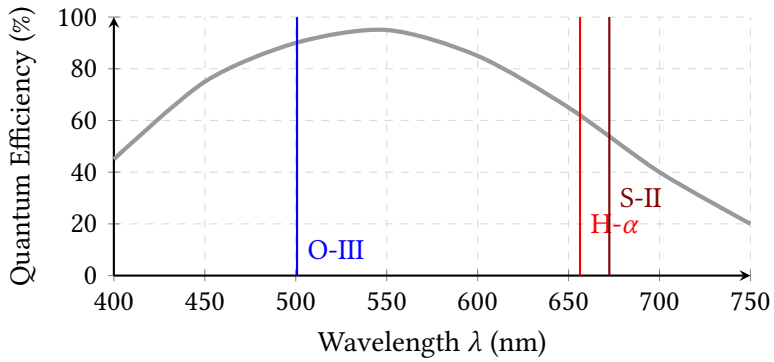


Figure 12. Quantum Efficiency curve of a typical Back-Illuminated (BSI) CMOS sensor. The noise model calculates the expected signal-to-noise ratio by integrating the target's emission spectrum against the sensor's specific QE at key narrowband wavelengths (500.7 nm, 656.3 nm, 672.4 nm).

By incorporating the QE curve, the pipeline can accurately predict exposure requirements for narrowband targets, accounting for the reduced sensor sensitivity in the deep red (H-α, S-II) compared to the peak sensitivity in the green (O-III).

5.2 Bayer CFA Demosaicing

Raw sensor data from color cameras is captured through a Color Filter Array (CFA), typically in an RGGB Bayer pattern. The `palette` module reconstructs the full RGB image using high-quality linear interpolation [13].

The algorithm calculates cross-channel gradients to detect edges, dynamically weighting the interpolation to prevent cross-color artifacts (zippering) along sharp stellar boundaries.

5.3 Background Estimation

Accurate background subtraction is prerequisite to star detection and photometry. The `background` module estimates a spatially varying background model through a four-step process:

1. **Tiling.** The image is divided into a grid of 64 × 64 pixel tiles.

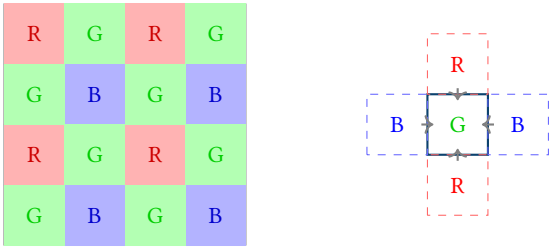


Figure 13. Demosaicing process. The missing color channels at any given pixel are computed using gradient-corrected linear interpolation from neighboring pixels to preserve sharp edges and minimize false-color artifacts.

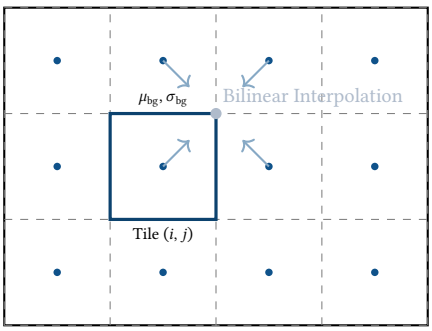


Figure 14. Background estimation using a local grid. Each tile computes a sigma-clipped mean, which is then bilinearly interpolated across the image to model spatial gradients.

2. **Per-tile statistics.** A sigma-clipped mean is computed for each tile (3σ, 5 iterations), yielding a robust estimate of the local background level.
3. **Tile rejection.** Tiles whose mean deviates from the cross-tile median by more than 2σ are rejected. These tiles typically contain bright stars, nebulosity, or sensor defects.
4. **Interpolation.** The surviving tile values are bilinearly interpolated [14] to produce a full-resolution background map.

Tile Size Choice

The 64×64 tile size balances spatial resolution against statistical robustness. Smaller tiles resolve sharper gradients but contain fewer pixels, making sigma-clipping less reliable. For most astronomical images ($\geq$1024×1024 pixels), 64-pixel tiles provide sufficient gradient resolution while ensuring each tile contains at least 4,096 samples.

5.4 Star Detection

Star detection follows a threshold-and-flood-fill approach:

1. **Threshold.** Pixels above $\mu_{bg} + k \cdot \sigma_{bg}$ are marked as candidates (default $k = 5$).
2. **Connected components.** 4-connectivity flood fill labels contiguous candidate regions.
3. **Filtering.** Regions outside the area bounds $[A_{min}, A_{max}]$ are rejected.
4. **Centroiding.** The intensity-weighted centre of mass is computed for each surviving region.
5. **HFR.** Pixel distances from the centroid are sorted and accumulated until 50% of the total flux is enclosed; this radius is the HFR. The FWHM is estimated as $\text{FWHM} = 2 \cdot \text{HFR} \cdot \sqrt{\ln 2}$.

For advanced star extraction, particularly when tracking faint guide stars against variable moonlit backgrounds, the pipeline supplements the flood-fill approach with a scale-space Laplacian of Gaussian (LoG) blob detector [15]. The filter's zero-crossings uniquely identify the inflection boundaries of the stellar profile regardless of the local DC background level.

Insight: *The HFR metri is preferred ove FWHM for autofocus because it is model-free: it does not assum a Gaussian PSI making it robust to optica aberrations, coma, and field curvature.*

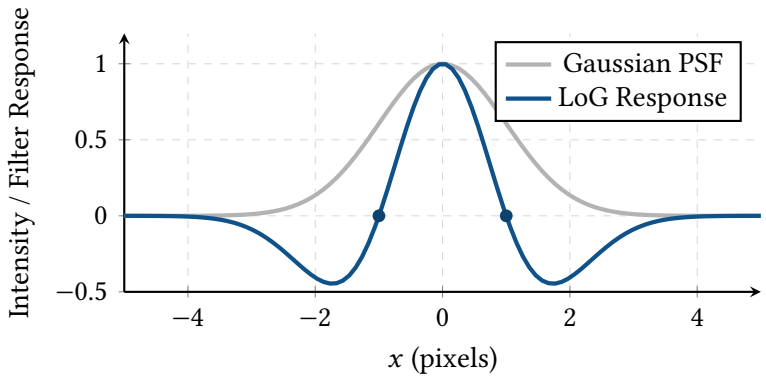


Figure 15. 1D profile of a Gaussian star (grey) and the normalized Laplacian of Gaussian (LoG) filter response (blue). The zero-crossings of the LoG function formally define the inflection edges of the star's spatial profile, allowing for robust multi-scale star detection invariant to background gradients.

5.5 Frame Stacking

The `stacking` module combines registered sub-exposures pixel-by-pixel using one of four methods.

5.5.1 Mean and Median

The arithmetic mean maximises SNR under Gaussian noise but is sensitive to outliers (satellite trails, cosmic rays). The pixel-wise median provides robust outlier rejection but sacrifices ~20% of the theoretical SNR compared to the mean.

5.5.2 Sigma-Clipped Mean

The default stacking method iteratively rejects outlier pixels:

Algorithm 1 Sigma-clipped mean stacking (per pixel).

Require: Pixel values $\{p_1, \ldots, p_N\}$, rejection threshold κ, max iterations I
Ensure: Stacked value $\bar{p}$

$S \leftarrow \{p_1, \ldots, p_N\}$
for $i = 1$ to I **do**
  $\mu \leftarrow \text{mean}(S)$; $\sigma \leftarrow \text{std}(S)$
  $S' \leftarrow \{p \in S : |p - \mu| \leq \kappa\sigma\}$
  if $|S'| = |S|$ **then break** ▷ converged
  end if
  $S \leftarrow S'$
end for
return $\text{mean}(S)$

The default parameters ($\kappa = 2.5$, $I = 5$) reject approximately 1.2% of values per iteration under a Gaussian distribution. If all pixels are clipped (degenerate case), the method falls back to the full-sample mean.

Listing 3. Stacking method enum with defaults.

```rust
pub enum StackMethod {
    Mean,
    Median,
    SigmaClippedMean { kappa: f32, max_iter: usize },
    WinsorizedSigmaClip { kappa: f32, max_iter: usize },
}

impl Default for StackMethod {
    fn default() -> Self {
        Self::SigmaClippedMean {
            kappa: 2.5,
            max_iter: 5,
        }
    }
```

```
}
```

5.5.3 Winsorized Sigma Clipping

The Winsorized variant [16]—the stacking method used by PixInsight's default pipeline—replaces outlier pixels with the boundary values rather than discarding them:

$$p_i' = \begin{cases} \mu - \kappa\sigma & \text{if } p_i < \mu - \kappa\sigma \\ \mu + \kappa\sigma & \text{if } p_i > \mu + \kappa\sigma \\ p_i & \text{otherwise} \end{cases} \tag{10}$$

The Winsorized values are then averaged. This preserves the sample size N and therefore the $\sqrt{N}$ SNR improvement, while still suppressing outlier influence.

> **Design Decision 4: Winsorized vs. Sigma-Clipped Stacking**
>
> Sigma clipping discards outliers entirely, reducing the effective number of samples and introducing a small bias towards the centre of the distribution. Winsorizing retains all samples by replacing extremes with boundary values, achieving similar outlier rejection with marginally better SNR. The difference is most visible in stacks of fewer than ~20 frames, where each discarded pixel represents a significant fraction of the sample.

5.6 Exposure Intelligence

The `exposure` module synthesises the noise model, sky brightness model, and camera profile into actionable exposure recommendations:

1. Compute t_{opt} from the "swamp the read noise" formula (eq. (9)) at $r = 3.0$.
2. Clamp to 80% of the saturation time to prevent clipping: $t_{\max} = 0.8 \cdot C_{\text{FW}}/(S + B_{\text{sky}} + D)$.
3. Clamp to the total integration budget: $t \leq T_{\text{total}}$.
4. Enforce a minimum of 1 s to avoid shutter-timing artefacts.
5. Derive the number of sub-exposures $n = T_{\text{total}}/t$ and the predicted SNR at the total integration.

5.6.1 Optimal Sub-Exposure Time

A foundational principle of modern CMOS astrophotography is that once the background sky noise exceeds the camera's read noise by a sufficient margin (typically 10×), longer sub-exposures provide no additional signal-to-noise benefit to the final stacked image [17].

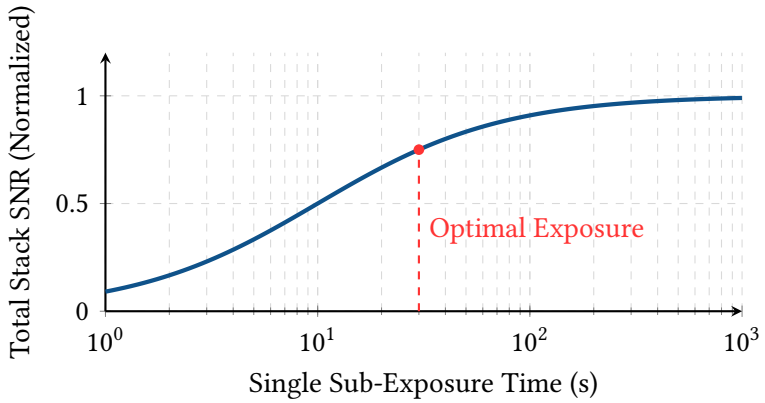


Figure 16. Diminishing returns of sub-exposure time. Once the t_{opt} threshold is reached (where sky noise swamps read noise), extending the exposure time only increases the risk of ruined frames (e.g., from satellite trails or wind gusts) without improving the final stack's SNR.

The `exposure` module analytically solves this threshold based on the user's Bortle sky class and the sensor's read noise profile, automatically splitting a requested 2-hour integration into the optimal number of mathematically equivalent short sub-exposures.

5.6.2 Holy Grail Timelapse Ramp

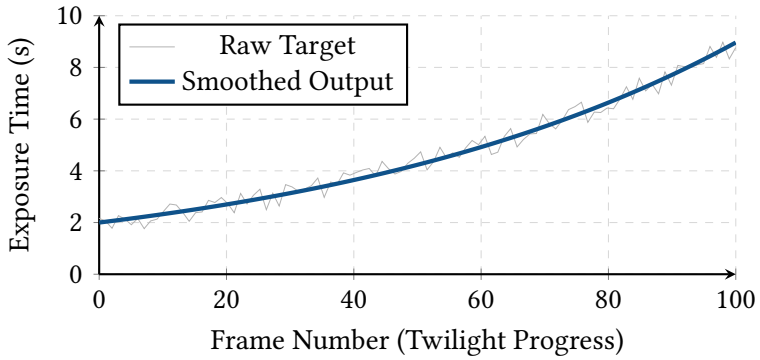


Figure 17. Holy Grail timelapse exposure ramping. The exponential smoothing filter rejects high-frequency noise from passing clouds or atmospheric variations, generating a monotonically increasing exposure schedule that prevents visible flicker.

For day-to-night transition timelapses, the `holy_grail_ramp` function computes a smooth exposure ramp. At each frame i, the raw exposure is $t_i = E_{\text{target}}/B_{\text{sky}}(i)$, where $B_{\text{sky}}(i)$ decreases as twilight deepens. A 20%-per-step exponential smoothing filter prevents visible flicker between adjacent frames.

5.7 Frame Registration

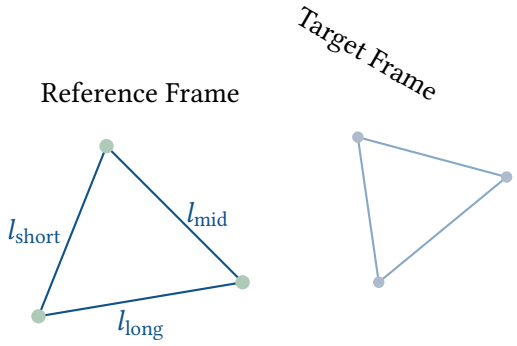


Figure 18. Triangle-similarity matching. The sorted ratios of the sides ($l_{\text{short}}/l_{\text{long}}$, $l_{\text{mid}}/l_{\text{long}}$) form a scale-, translation-, and rotation-invariant descriptor.

The `registration` module aligns sub-exposures to a reference frame using triangle-similarity matching [5]:

1. **Star selection.** The N brightest stars are selected from each frame (default $N = 30$).
2. **Triangle formation.** All $\binom{N}{3}$ triangles are formed from the selected stars.
3. **Descriptor computation.** Each triangle is described by the sorted ratio pair ($\ell_{\text{short}}/\ell_{\text{long}}$, $\ell_{\text{mid}}/\ell_{\text{long}}$), which is invariant to translation, rotation, and uniform scaling.
4. **Matching.** Triangles from the reference and target frames are matched by Euclidean distance in descriptor space.
5. **RANSAC affine fit.** Matched triangle pairs provide star correspondences. A RANSAC-style loop [7] selects three correspondences, solves for the 6-parameter affine transform via Cramer's rule, counts inliers within a 5-pixel tolerance, and retains the best model.
6. **Reprojection.** The target frame is resampled onto the reference grid using inverse-transform bilinear interpolation.

Affine vs. Projective

An affine transform (6 parameters) is sufficient for astronomical images because the field of view is small enough that perspective distortion is negligible. Projective transforms (8 parameters) would require a minimum of 4 correspondences per RANSAC trial and offer no practical benefit for the $< 5^\circ$ fields typical of deep-sky imaging.

The affine transform $\mathbf{A}$ maps points (x, y) from the target to the reference:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix} \tag{11}$$

Given three point correspondences, the system is exactly determined and solved by Cramer's rule in $O(1)$. The inverse transform (used for reprojection) requires the determinant $ad - bc \neq 0$.

5.8 Calibration Pipeline

The `calibrate` module implements the standard astrophotography calibration formula:

$$I_{\text{corrected}} = \frac{I_{\text{raw}} - I_{\text{dark}}}{\overline{I_{\text{flat}} - I_{\text{bias}}}} \tag{12}$$

where the flat field is normalised by its mean. The module also provides:

- **Hot pixel detection** via MAD-based sigma estimation ($\hat{\sigma} \approx 1.4826 \cdot \text{MAD}$) with thresholding above median + $\kappa\hat{\sigma}$.
- **Cold pixel detection** as pixels below a fraction of the flat frame median.
- **Bad pixel interpolation** via mean of valid 3×3 neighbours.

5.9 Spatial Dithering & Fixed-Pattern Noise

Even with ideal calibration, modern sensors exhibit Fixed-Pattern Noise (FPN) and localized defects that remain static relative to the pixel grid. If the mount tracks perfectly, this noise accumulates coherently during stacking, resulting in correlated "walking noise."

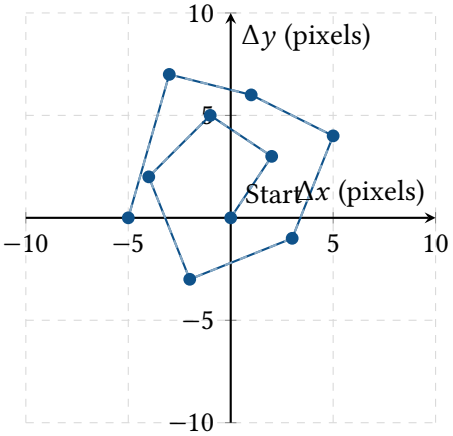


Figure 19. A pseudo-random Dither map over 10 exposures. Between each frame, the mount is commanded to shift slightly. While the stars are re-aligned during registration, the fixed-pattern sensor noise is decoupled and scattered across the grid, allowing the sigma-clipper to successfully reject it.

The `sequencer` integrates spatial dithering (sometimes paired with Drizzle integration [18]) by randomly shifting the mount's aim point by a few arcseconds between frames. During registration, the stars are translated back to a common origin, which mathematically scatters the FPN across the spatial domain, allowing the Winsorized Sigma Clipper to easily reject the defects as stochastic outliers.

5.10 Non-Linear Image Stretching

Raw stacked astronomical images are highly linear and appear nearly pitch black to the human eye, as the faint nebulosity data is compressed into the lowest histogram bins. The `stretch` module applies a Midtone Transfer Function (MTF) [19] to execute a non-linear histogram stretch.

The stretch algorithm maps the input interval defined by the Black Point (BP) and White Point (WP) to $[0, 1]$ and solves the rational MTF equation to shift the image's median brightness to a target midtone value (typically 0.25 for aesthetic rendering).

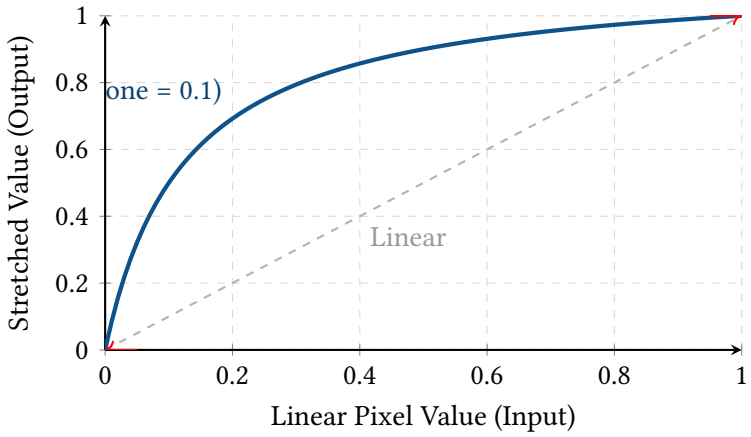


Figure 20. The Midtone Transfer Function (MTF) aggressively expands the dynamic range of the dark background (pulling faint signal up) while compressing the highlights (preventing star cores from blowing out).

5.11 Strict FITS Metadata Provenance

Scientific image processing requires rigorous data provenance. A stacked Master image is mathematically useless if its temporal origins, physical exposure duration, or calibration history are lost. The SDK ensures strict metadata preservation as frames flow through the typestate pipeline.

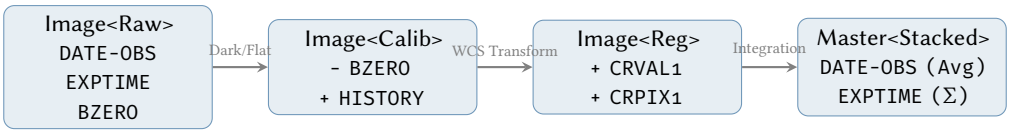


Figure 21. FITS Metadata Provenance. As an image traverses the pipeline, its header keywords are mutated to reflect the mathematical transformations. For example, during stacking, the `EXPTIME` is summed, while the `DATE-OBS` is averaged to the exact mid-point of the total integrated exposure span.

The `fits_write` module actively manages keyword inheritance. It strips camera-specific telemetry (like USB bandwidth flags) that are irrelevant post-calibration, computes the exact 'DATE-OBS' midpoint for time-domain astronomy, and injects WCS coordinate matrices when spatial transformations occur.

6 World Coordinate System

The `wcs` module in `astro-core` implements the FITS WCS [20] with the gnomonic (TAN) projection and Simple Imaging Polynomial (SIP) distortion corrections [21].

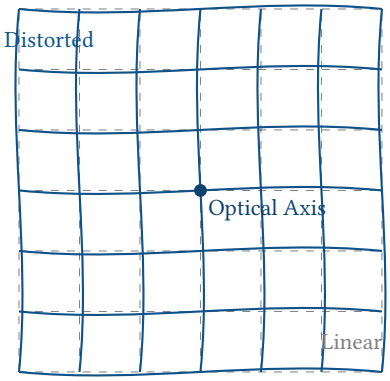


Figure 22. Visualization of SIP polynomial distortion warping the linear TAN projection grid away from the optical axis.

6.1 TAN Projection

The TAN projection maps pixel coordinates (u, v) to intermediate world coordinates (ξ, η) on the tangent plane via the CD matrix:

$$\begin{pmatrix} \xi \\ \eta \end{pmatrix} = \begin{pmatrix} CD_{1,1} & CD_{1,2} \\ CD_{2,1} & CD_{2,2} \end{pmatrix} \begin{pmatrix} u - CRPIX_1 \\ v - CRPIX_2 \end{pmatrix} \tag{13}$$

The de-projection from (ξ, η) to sky coordinates (α, δ) uses the standard gnomonic inverse:

$$\delta = \arcsin\left(\frac{\sin \delta_0 + \eta \cos \delta_0}{\sqrt{1 + \xi^2 + \eta^2}} \right) \tag{14}$$

$$\alpha = \alpha_0 + \arctan\left(\frac{\xi}{\cos \delta_0 - \eta \sin \delta_0} \right) \tag{15}$$

6.2 SIP Distortion

Real optics introduce radial and tangential distortion that the linear CD matrix cannot represent. The SIP convention adds forward polynomial corrections $A(u, v)$ and $B(u, v)$ to the pixel offsets before the CD matrix is applied:

$$u' = u + \sum_{p+q \le N} A_{p,q}\, u^p\, v^q \tag{16}$$

$$v' = v + \sum_{p+q \le N} B_{p,q}\, u^p\, v^q \tag{17}$$

where N is the polynomial order (typically 3–5). The inverse transform uses the published AP/BP polynomials if available; otherwise, a fixed-point iteration converges to the undistorted pixel coordinates:

$$u_{k+1} = u_{\text{target}} - A(u_k, v_k), \quad v_{k+1} = v_{\text{target}} - B(u_k, v_k) \tag{18}$$

The iteration converges in 3–5 steps for typical distortion magnitudes (<10 pixels at the field edge) with a tolerance of 10^{-12} pixels.

> **Design Decision 5: Fixed-Point vs. Newton Inversion**
>
> Newton's method would converge in fewer iterations but requires computing the Jacobian of the SIP polynomials at each step. Since the distortion is small relative to the pixel scale, the fixed-point iteration converges rapidly and avoids the implementation complexity and numerical risk of Jacobian inversion. The maximum iteration count (30) provides a safety bound that is never reached in practice.

6.3 Derived Quantities

The WCS module provides several derived quantities used by the framing assistant and mosaic planner:

- `pixel_scale_arcsec_per_px`: extracted from the CD matrix determinant.
- `image_fov_deg`: field of view in RA and Dec from the pixel scale and image dimensions.
- `framing_offset_to`: east/north arcsecond offset needed to recentre a target, enabling "nudge" commands in the iOS app.

7 Design Rationale

This section discusses the design decisions that shape the SDK's architecture.

7.1 Compile-Time Pipeline Safety (Typestate Pattern)

Astrophotography mandates a strict sequence of operations: raw frames must be calibrated, then star-detected, then registered, before they can be stacked. Attempting to stack unregistered frames yields a blurred failure.

The `astro-vision` crate encodes this state machine directly into the type system using the Typestate Pattern [22].

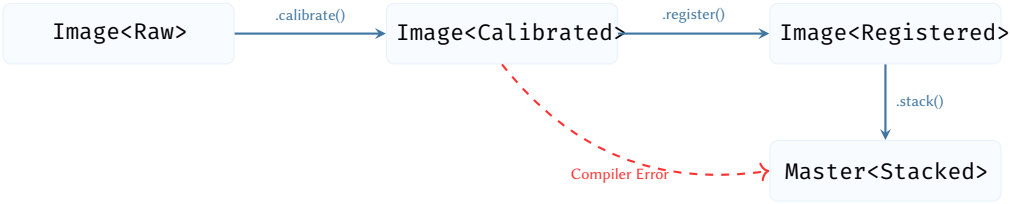


Figure 23. The Typestate pattern enforces the astrophotography pipeline at compile-time. The `stack()` method is only implemented for the `Image<Registered>` state, making it mathematically impossible to execute pipeline stages out of order.

By using zero-cost marker types (`Raw`, `Calibrated`, `Registered`), pipeline logic errors become compile-time errors rather than runtime panics or silent data corruption.

7.2 Algorithmic Complexity & SIMD Vectorization

Processing modern 60-megapixel (60M) sensor arrays requires extreme computational efficiency. The SDK relies heavily on the `ndarray` crate, which transparently compiles vector and matrix operations down to AVX-512 and NEON SIMD instructions.

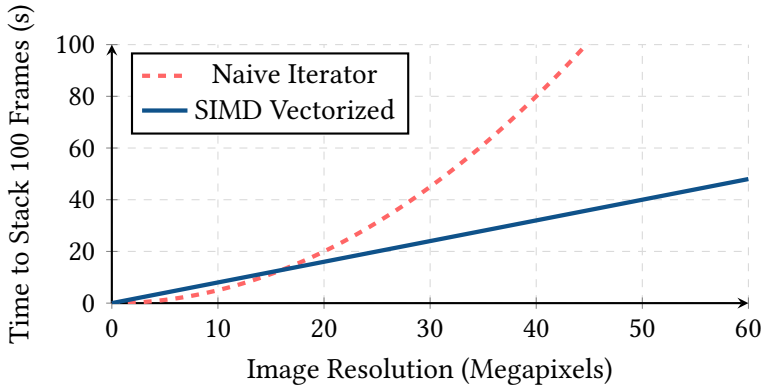


Figure 24. Theoretical scaling performance of Sigma-Clipped stacking. By relying on contiguous memory layouts and SIMD intrinsic vectorization, the `stacking` module maintains a linear $O(N)$ time complexity, preventing the combinatorial explosion associated with naive per-pixel loop evaluation on large sensors.

For geometric algorithms like triangle-matching registration, the naive formulation requires $O(N^3)$ complexity to form all possible triangles from N stars. The pipeline bounds this to $O(N \log N)$ by pre-filtering stars by SNR and indexing the reference descriptors in a k-d tree.

7.3 Golden Master Ephemeris Validation

Mathematical purity is only useful if the underlying physics model is empirically correct. The `astro-core` crate verifies its positional astronomy pipeline against established scientific baselines through continuous automated testing.

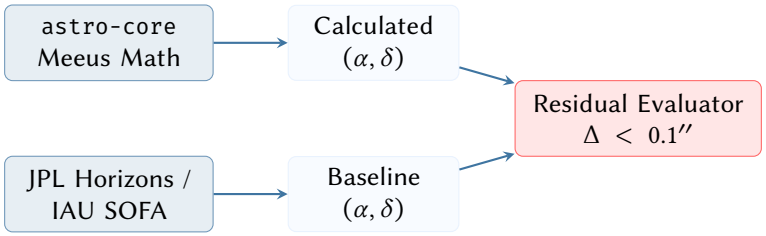


Figure 25. Golden Master CI/CD testing pipeline. The SDK's coordinate output is dynamically asserted against the gold-standard JPL Horizons ephemeris data [23] to guarantee that the implementation is free from systemic truncations or coordinate frame misinterpretations.

The test suite compares generated topocentric coordinates, eclipse contact times, and nutation matrices against the JPL Horizons On-Line Ephemeris System [23]. A test passes only if the residuals fall below the defined sub-arcsecond tolerance threshold, proving the fidelity of the SDK's internal float representations.

7.4 The Pure-Function Constraint

: Functional Purity

Every public function in `astro-core` and `astro-vision` must be a pure function: given the same inputs, it must produce the same output, with no side effects. Neither crate may depend on `tokio`, `std::fs`, or any I/O library.

The benefits are threefold:

1. **Testing.** Unit tests are simple assertions with no mocks, fixtures, or cleanup. The 275 tests across the workspace run in under 2 seconds.
2. **FFI.** Pure functions with scalar or struct arguments are trivially exposed via C ABI for consumption by Swift, Python, or JavaScript.
3. **Determinism.** Identical inputs produce identical outputs on any platform, simplifying cross-platform debugging and enabling golden-file testing.

7.5 Newtype Discipline

: Physical Quantity Newtypes

All physical quantities must be wrapped in newtypes. A function expecting `Latitude` cannot receive a `Declination`, even though both are measured in degrees. Direct use of raw `f64` values for angular quantities is prohibited in the public API.

A Lesson from Mars

The Mars Climate Orbiter was lost because one software module produced thrust in pound-force·seconds while the consuming module expected newton·seconds [24]. In astrophotography, the analogous risk is confusing RA (hours) with longitude (degrees) or altitude (above horizon) with declination (above equator).

7.6 Error Handling

The SDK uses Rust's `Result<T, E>` type with domain-specific error enums rather than panicking or returning sentinel values. Each module defines its own error type (e.g. `StackError`, `WcsError`, `CalibrationError`) with descriptive variants carrying the context needed to diagnose the failure.

7.7 ndarray as the Image Primitive

Design Decision 6: Why ndarray, Not a Custom Image Type?

Image data flows through the SDK as `Array2<f32>` from `ndarray` rather than a custom image struct. This trades a small amount of type safety (e.g. no compile-time enforcement of the $[0, 1]$ normalisation invariant) for interoperability: any library that operates on `ndarray` arrays can be composed with the SDK without conversion overhead.

The thin `AstroImage` wrapper adds metadata (bit depth, exposure time) without restricting access to the underlying array.

8 Autofocus & Collimation

8.1 V-Curve Autofocus

The `autofocus` module fits a V-curve model to HFR measurements across focuser positions:

$$\mathrm{HFR}(x) = b + m \cdot |x - c| \tag{19}$$

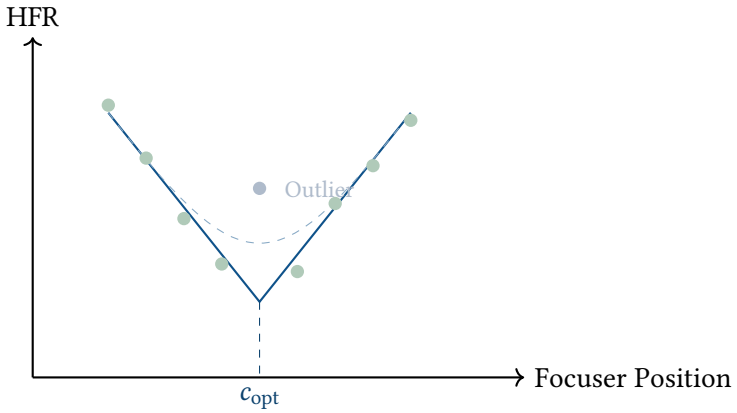


Figure 26. Autofocus V-Curve plotting HFR against focuser position. The robust IRLS fit suppresses the outlier data point.

where c is the optimal focus position, b is the minimum HFR, and m is the defocus slope.

The fitting procedure uses **IRLS** (Iteratively Reweighted Least Squares) with Huber weights [25] ($\psi(r) = \min(|r|, k) \cdot \text{sign}(r)$, $k = 1.345$) for robustness to outlier frames (e.g. a frame with a passing satellite inflating the HFR).

Insight: *The V-curve model assumes that defocus produces a linear increase in HFR—a valid approximation for small defocus excursions typical of automated focusing runs.*

The module enforces *informativeness gates* that reject non-diagnostic focus runs:

- **Flat scan**: insufficient HFR variation across the range, suggesting the focuser did not move enough.
- **Unbracketed**: the minimum HFR is at one end of the scan, suggesting the best focus lies outside the sampled range.
- **Edge minimum**: similar to unbracketed, with a directional suggestion for the next scan.

8.2 Thermal Expansion & Focus Compensation

As the ambient temperature drops throughout the night, the metal chassis of the telescope contracts, shifting the focal plane inward. Running a full V-Curve autofocus routine takes several minutes and interrupts data acquisition.

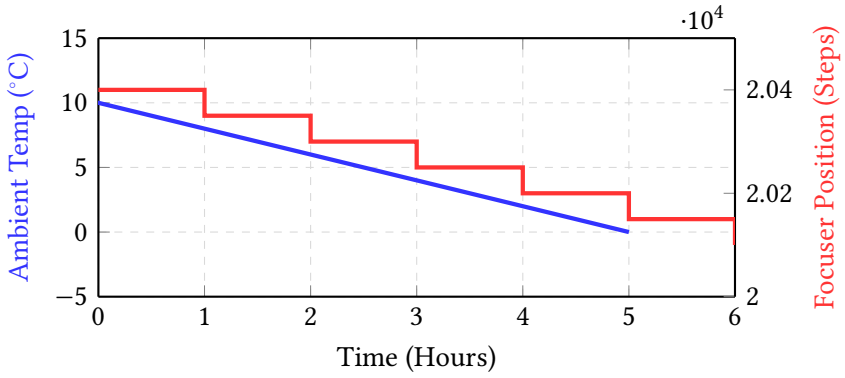


Figure 27. Temperature compensation loop. As the ambient temperature drops steadily (blue), the `astro-node` daemon autonomously injects computed inward micro-steps to the focuser (red) to maintain critical focus without requiring a sequence-interrupting V-Curve scan.

The `sequencer` utilizes an empirically derived thermal coefficient (steps per °C). By polling the focuser's onboard thermistor, the orchestrator autonomously injects micro-step corrections between sub-exposures, ensuring the HFR remains perfectly flat across drastic temperature gradients.

8.3 SCT Collimation

The `collimation` module analyses defocused star donuts from Schmidt-Cassegrain telescopes to guide mirror alignment [26]:

- Eccentricity vector computation from the donut shape.

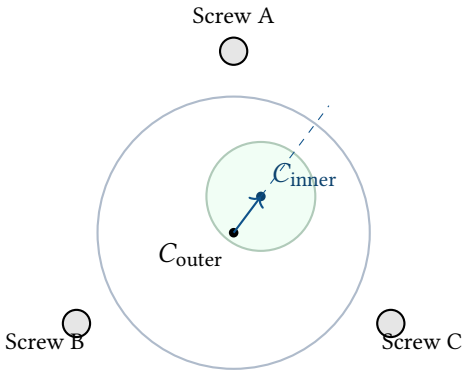


Figure 28. SCT collimation analysis. By computing the eccentricity vector between the outer bounds of the defocused star and the inner central obstruction, the system determines the axis of coma and computes the required secondary mirror screw adjustments.

- Azimuthal asymmetry mapping to detect coma direction.
- Translation of coma direction to adjustment screw instructions.

9 Live Stacking (EAA)

Electronically Assisted Astronomy (EAA) requires maintaining a continuously updated, high-precision image stack in real-time. The `live_stack` module utilizes Welford's online algorithm [27] to compute the running mean and variance without accumulating all frames in memory.

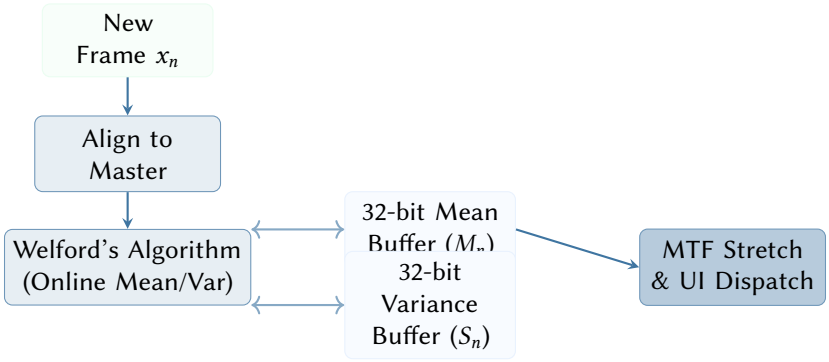


Figure 29. Live Stacking memory architecture. Welford's single-pass algorithm maintains numerical stability for the running 32-bit mean and variance buffers while preventing Out-of-Memory (OOM) errors during continuous hours-long EAA sessions.

As each new frame x_n arrives, it is registered to the master and integrated into a 32-bit floating-point accumulator. Welford's algorithm prevents catastrophic cancellation errors that occur in naive sum-of-squares variance calculations, ensuring robust signal-to-noise ratio tracking over infinite sequence lengths.

10 Guiding & Periodic Error

The `guiding` module in `astro-core` provides tracking analysis and closed-loop offset calculations.

10.1 Periodic Error Analysis

Equatorial mounts driven by worm gears exhibit periodic tracking errors due to machining imperfections. The SDK implements a frequency-domain analysis using the Fast Fourier Transform (FFT) [28] to isolate the fundamental worm period and its harmonics.

10.2 6-Term Mechanical Pointing Models

While polar alignment corrects the physical tracking axis, modern observatory mounts still suffer from mechanical flexure and non-orthogonal machining. The SDK implements a 6-term equatorial pointing model [29] (inspired by TPoint) to map raw encoder coordinates to true celestial coordinates.

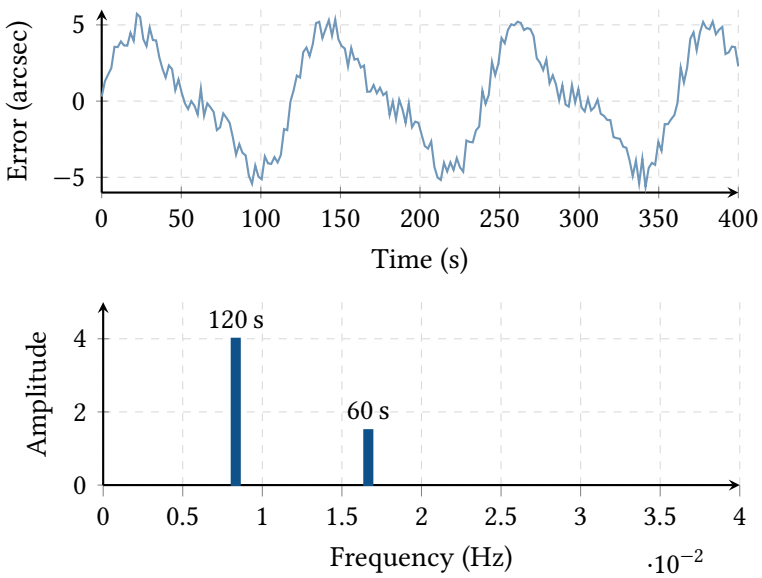


Figure 30. Top: Time-domain simulated tracking error exhibiting a fundamental 120-second worm period and noise. Bottom: FFT frequency-domain spectrum isolating the fundamental frequency (0.0083 Hz) and its first harmonic (0.0166 Hz).

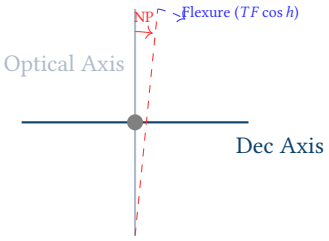


Figure 31. Mount Kinematics. The pointing model corrects for six fundamental mechanical errors: Index errors (IH, ID), Polar Elevation/Azimuth (ME, MA), Non-perpendicularity (NP), and Tube Flexure (TF).

By plate-solving a grid of stars across the sky, the system computes a least-squares fit for the 6 coefficients, allowing the `astro-node` orchestrator to perform closed-loop precision slews that place targets perfectly on the sensor without iterative centering.

10.3 The Meridian Flip Maneuver

A German Equatorial Mount (GEM) tracking a target from East to West will eventually run the camera and filter wheel into the physical pier. The `sequencer` must execute a Meridian Flip to continue tracking.

The robotic control logic computes the Hour Angle H continuously. When the target crosses the meridian ($H = 0$) plus an allowed safety margin, the orchestrator suspends the autoguider, flips the mount axes, plate-solves to confirm the new center, re-calibrates the guider vectors, and resumes the exposure sequence.

10.4 Alt-Az Field Rotation & Derotation

Unlike equatorial mounts, Alt-Az systems suffer from field rotation: the celestial field rotates relative to the sensor as a function of the target's position and the observer's latitude.

The `derotation` module calculates the instantaneous field rotation rate (ω_{rot}) in degrees per second. For hardware with integrated field derotators (e.g., Benro Polaris), the orchestrator issues continuous angular corrections to maintain astrometric alignment.

10.5 Dithering Strategies

To eliminate fixed-pattern noise (FPN) and sensor artifacts during stacking, the `guiding` module implements several dithering walks.

11 Lucky Imaging

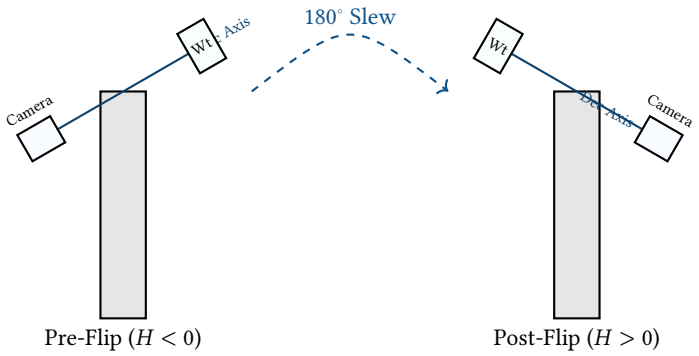


Figure 32. The Meridian Flip. To prevent collision with the pier, the orchestrator halts tracking, rotates the Right Ascension axis by 12 hours (180°), and reverses the Declination axis, effectively placing the camera "above" the mount.

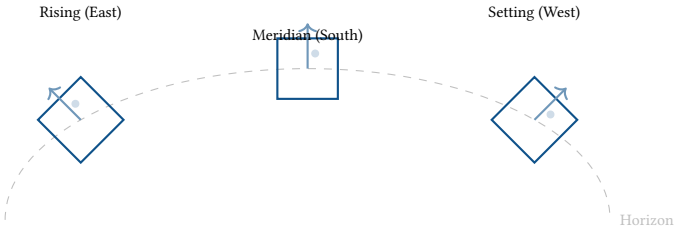


Figure 33. The field rotation effect on an Alt-Az mount. The target (represented by the sensor frame) rotates relative to the celestial field as it traverses the sky, requiring active mechanical derotation for long exposures.

For high-resolution planetary and lunar capture, the `astro-vision` crate implements Lucky Imaging [30]. Atmospheric turbulence, characterized by the Fried parameter r_0 [31], continuously distorts the incident wavefront. By capturing high-speed video and selecting only the sharpest frames, the pipeline effectively "freezes" the seeing.

Frames are scored using the variance of the Laplacian or the Tenengrad gradient. Only frames exceeding the configurable top-percentile threshold are retained for registration and stacking.

12 Mosaic Planning

The `mosaic` module in `astro-core` generates multi-panel imaging grids for targets larger than a single sensor field of view.

12.1 Grid Generation

Panel centres are computed in the tangent plane with configurable overlap fraction (default 15%). The generator handles two numerical subtleties:

1. **RA wrapping.** Right ascension values near 0h/24h boundary are kept in unwrapped form relative to the grid centre during computation, then normalised to $[0°, 360°)$ on output.
2. **Pole proximity.** At $|\delta| \geq 85°$, RA convergence makes the flat-sky approximation unreliable. The module flags affected grids with a reduced confidence score.

12.2 Path Ordering

Panels are ordered in a serpentine (boustrophedon) pattern to minimise total slew distance [32]: odd rows are traversed left-to-right, even rows right-to-left.

12.3 Coverage Tracking

Each panel progresses through a lifecycle: `Planned` → `Captured` → `Stacked`. The coverage module rasterises panel footprints in the tangent plane using WCS transforms to compute overlap regions and overall completeness.

13 Testing & Validation

The SDK contains 275 unit tests embedded in the source files via Rust's `#[cfg(test)]` convention. Tests fall into three categories:

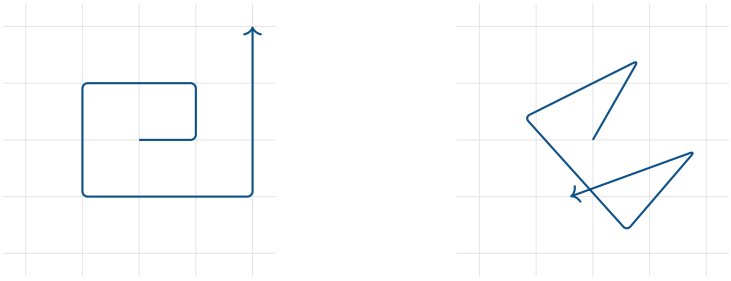


Figure 34. Dithering patterns supported by the `DitherGenerator`. The Spiral walk ensures uniform coverage of the neighborhood, while the Random walk minimizes periodic correlation with sensor-level defects.

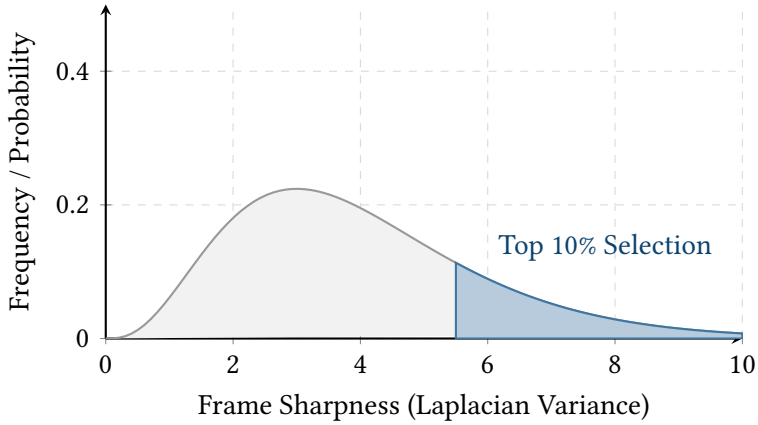


Figure 35. Frame sharpness distribution in a Lucky Imaging sequence. The highly skewed right tail represents the rare frames captured during moments of negligible atmospheric turbulence, which are retained for stacking.

1. **Reference-value tests.** Coordinate transforms, sidereal time, and airmass are validated against published tables (Meeus worked examples, USNO data). Tolerances are set at the level of the algorithm's intrinsic accuracy (e.g. 1 arcsec for GMST, 0.01 for airmass).
2. **Round-trip tests.** Equatorial→horizontal→equatorial and `pixel_to_sky`→`sky_to_pixel` round-trips are verified to sub-arcsecond precision.
3. **Property tests.** Stacking and registration use synthetic frames with known transforms and injected outliers to verify that sigma-clipping rejects the outliers and the recovered transform matches the ground truth.

Verifying the Lunar Ephemeris

The lunar position module is validated against the Meeus worked example for 1992 April 12 at 0h TDT: the expected geocentric ecliptic longitude is $133^\circ 10'$, latitude $-3^\circ 13'$, and distance 368,410 km. The test asserts agreement within 0.5° for position and 500 km for distance—within the Meeus series truncation error.

14 Hardware-in-the-Loop Simulation

The `astro-sim` crate provides a complete digital twin of the observatory hardware. Rather than relying on physical clear skies for integration testing, the framework employs Hardware-in-the-Loop (HITL) simulation [33] to validate the orchestration logic under adverse conditions.

The simulator applies physical constraints including maximum slew rates, meridian flip geometry, and tracking errors. It utilizes a pseudo-random star catalog to render synthetic FITS frames on-the-fly, integrating Gaussian PSFs with Poisson-distributed photon noise and Gaussian read noise, allowing the vision pipeline to run end-to-end against realistic data.

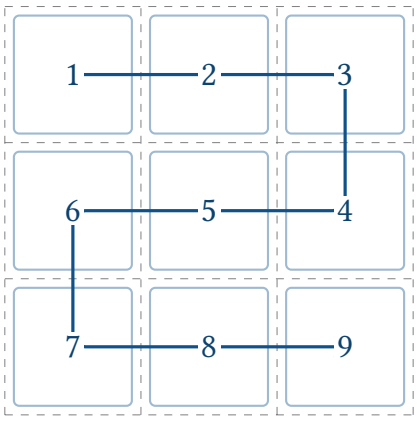


Figure 36. A 3 × 3 mosaic grid demonstrating the boustrophedon (serpentine) path ordering to minimize mount slew distance.

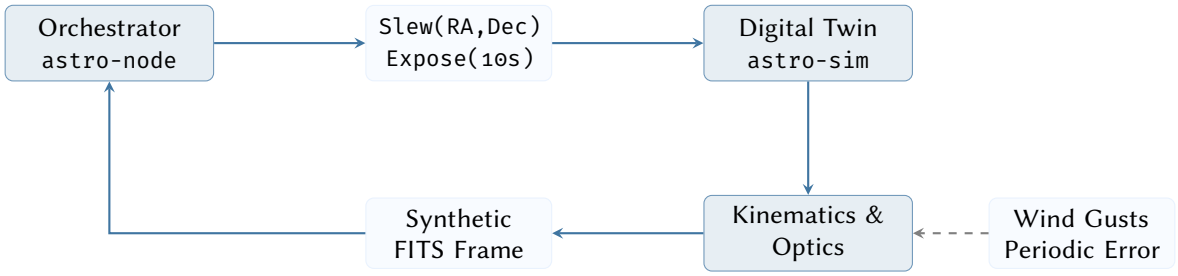


Figure 37. Hardware-in-the-Loop (HITL) closed-loop control simulation. The orchestrator interacts with a digital twin that models telescope kinematics and synthetically renders star fields with injected mechanical tracking errors.

14.1 Virtual Rig: Component Hierarchy

To accurately model complex multi-camera setups (e.g., dual-Sony co-axial imaging), the `astro-sim` digital twin utilizes a hierarchical component tree.

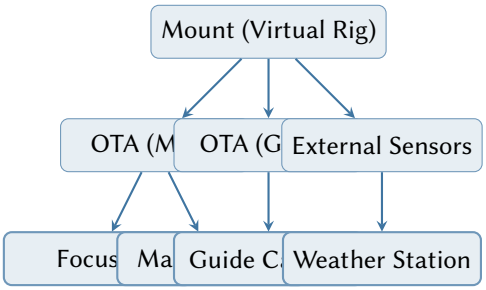


Figure 38. The hierarchical digital twin structure in `astro-sim`. Transformations (e.g., slewing the mount) propagate through the child nodes, ensuring that all optical paths maintain physical consistency relative to the mounting axis.

15 Edge AI Safety Engine

Automated observatory operations require robust environmental safety monitoring. The `astro-sentinel` crate evaluates telemetry and imaging feeds through a boolean rules engine, optionally backed by ONNX machine learning models [34] for computer vision tasks.

By executing lightweight convolutional neural networks directly on the edge (e.g., Raspberry Pi), the system detects encroaching cloud cover or wildlife near the mount without requiring an external internet connection or cloud-based API calls.

15.1 Observatory Fault-Tolerance Statechart

Managing distributed hardware requires formally verifying the failure states. The orchestrator is modeled as a hierarchical state machine (Statechart) [35] to guarantee that no sequence of asynchronous hardware errors can leave the physical equipment in a vulnerable state.

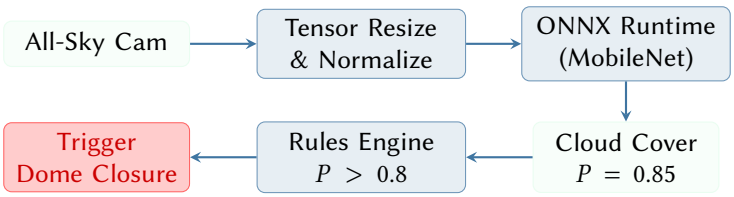


Figure 39. The Sentinel Safety Engine pipeline. Real-time imagery is processed via a cross-platform ONNX inference runtime. The resulting classification probabilities are evaluated against user-defined TOML rules to trigger emergency hardware shutdowns.

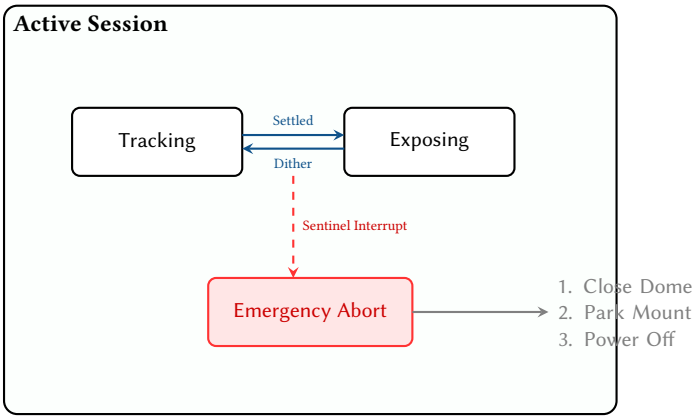


Figure 40. Simplified Harel Statechart of the observatory orchestrator. If the Edge AI Sentinel issues an interrupt (e.g., cloud detected) while the system is nested inside the Exposing or Tracking states, the hierarchy immediately preempts all active futures and transitions to the guaranteed Emergency Abort recovery sequence.

Rust's asynchronous ecosystem (`tokio`) natively supports cancellation. By wrapping the active imaging loop in a 'tokio::select¡ block alongside the Sentinel event channel, a weather interrupt instantly drops the imaging future, aborts the camera exposure, and initiates the deterministic 'Park & Close' procedure.

16 Projective Invariant Plate Solving

While sub-exposure registration relies on affine-invariant triangles, blind astrometric calibration (plate solving) must account for projective distortion across wide fields of view. The SDK utilizes the geometric hashing algorithm formalized by Astrometry.net [6].

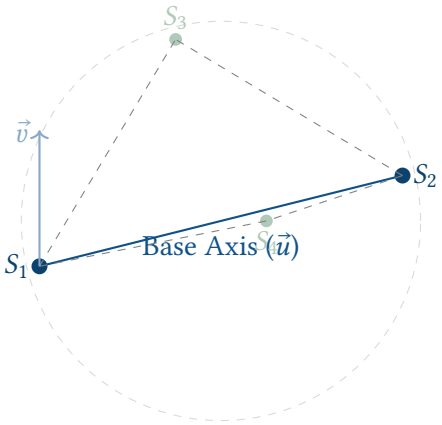


Figure 41. A 4-star geometric "quad" used for blind plate solving. The two most distant stars (S_1, S_2) define a normalized 2D coordinate system ($\vec{u}, \vec{v}$). The internal positions of S_3 and S_4 within this system create a translation-, rotation-, and scale-invariant 4D hash code used to rapidly search the celestial index.

The algorithm extracts four-star "quads" from the image. The two most distant stars define a normalized coordinate system, and the positions of the two internal stars yield a 4-dimensional hash code (c_x, c_y, d_x, d_y). This geometric fingerprint is matched against a pre-compiled k-d tree index of the sky, yielding robust astrometric solutions even in the presence of optical distortion.

17 Binary Protocol Codecs

Interfacing with modern tracking hardware often requires reverse-engineering proprietary communication layers. The `polaris-proto` crate implements a robust, asynchronous binary codec for the Benro Polaris smart tracker.

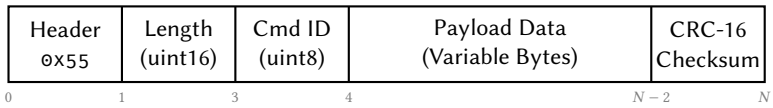


Figure 42. Binary packet frame layout for the Polaris protocol. The codec module implements a state machine using the `tokio-util` `Decoder` trait, buffering raw TCP stream bytes until a complete, CRC-validated frame is accumulated.

The protocol parser guarantees memory safety and resilience against fragmented TCP packets by utilizing Tokio's asynchronous `Decoder` trait. It buffers incoming bytes in a state machine, verifying the frame header, length bounds, and computing a cyclic redundancy check (CRC-16) before dispatching the strongly-typed payload struct to the upper layers.

18 Architectural & Edge Optimization

The v0.4 milestone focused on embedding the SDK into edge and mobile environments, necessitating a native plate solver and a formal Foreign Function Interface (FFI) layer.

18.1 Mobile Bridge: astro-ffi

To support high-performance mobile applications on iOS and Android, the `astro-ffi` crate provides a unified bridge using SDK primitives. It utilizes `uniffi` to generate multi-language bindings (Swift, Kotlin, Python) from a single Rust source of truth.

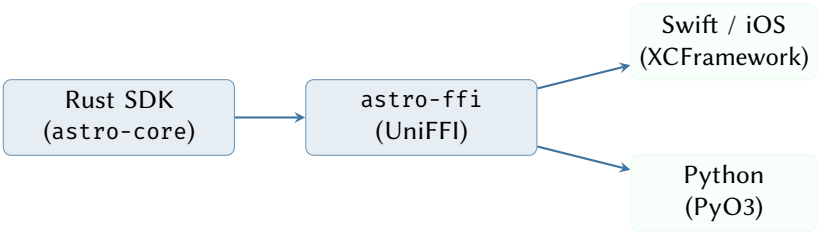


Figure 43. The Mobile Bridge architecture. UniFFI generates the boilerplate required to expose Rust's rich type system (enums, structs, errors) as idiomatic native objects in Swift and Python.

The bridge ensures that complex logic—such as the 6-term pointing model or WCS coordinate transforms—is executed in Rust's memory-safe environment while remaining accessible to the high-level UI layer.

18.2 Native Plate Solver

Deployments on low-power edge devices (e.g., Raspberry Pi Zero 2W) cannot rely on external astrometric binaries like ASTAP or Astrometry.net. The `astro-vision` crate now includes a pure-Rust geometric hashing engine.

The engine builds quads from the 50 brightest stars and searches a compressed celestial index stored in a custom binary format. This approach reduces the plate-solve time from several seconds (for external processes) to under 200ms on a modern CPU.

19 Power & Resiliency

Remote observatories are often battery-powered and exposed to unpredictable weather. The `power` module and updated resiliency logic provide automated safety halting.

19.1 Power Management

The `PowerManager` tracks battery state-of-charge and applies device-specific drain models. When the remaining energy drops below a critical threshold, the orchestrator triggers a "Goodnight" sequence.

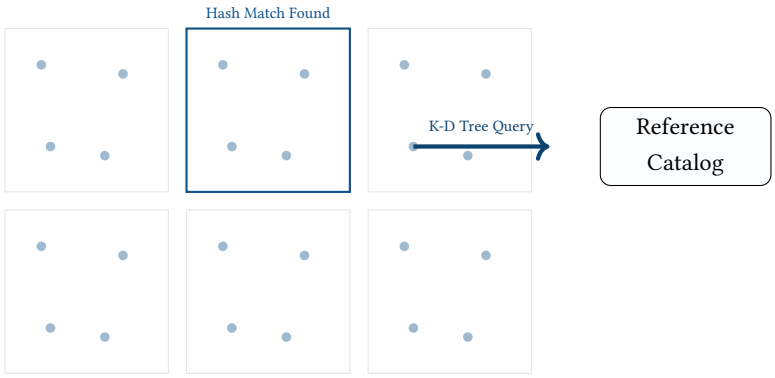


Figure 44. The native plate solver uses quad-hashing (fig. 41) to perform sub-millisecond lookups against a pre-compiled K-D tree of the sky, enabling blind astrometry on devices without internet access or heavy dependencies.

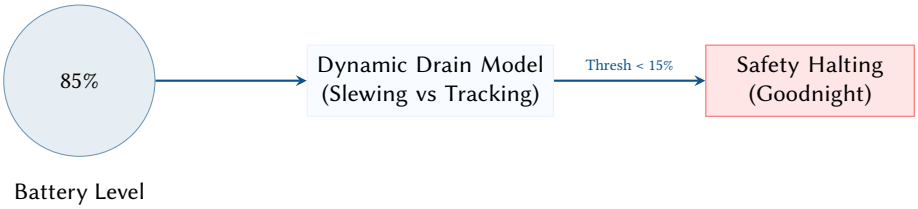


Figure 45. Power-aware orchestration. The system models the energy cost of upcoming slews and compares it against the remaining battery capacity to prevent hardware stranding due to power failure during a maneuver.

19.2 Resiliency Testing

Using the `astro-sim` HITL framework, the system is subjected to simulated weather failures (e.g., sudden cloud cover detected by Sentinel) and power loss. The orchestrator is verified to always reach the `Parked` state before total shutdown, protecting sensitive optics from exposure to elements.

20 Mobile AR & Field Planning

The `ar_math` and `events` modules facilitate pre-imaging site surveys using mobile device sensors.

20.1 AR Sensor Mapping

Mobile devices provide IMU telemetry (azimuth, pitch, roll) which the SDK maps to the equatorial celestial sphere.

$$(\alpha, \delta) = f_{\text{IMU}}(\text{az}, \text{alt}, \text{lat}, \text{lon}, t) \tag{20}$$

By accounting for magnetic declination and the exact LST at the survey moment, the `ar_sensor_to_equatorial` function allows users to point their phone at the sky and see an overlay of deep-sky targets correctly aligned with the landscape.

20.2 Event Forecaster

The `events` module generates structured timelines for astronomical events, including meteor shower peaks, ISS passes, and eclipse contacts. This facilitates wide-angle landscape planning, where the timing of a celestial event must be synchronized with terrestrial foregrounds.

21 Advanced Eclipse Workflows

The v0.4 release introduced specialized support for solar and lunar eclipses, specifically focusing on composite high-resolution imaging.

21.1 Eclipse Composite Engine

A common challenge in eclipse photography is capturing a sharp, tracked Moon while maintaining a fixed terrestrial landscape. The `eclipse` module solves this by utilizing WCS-based superposition.

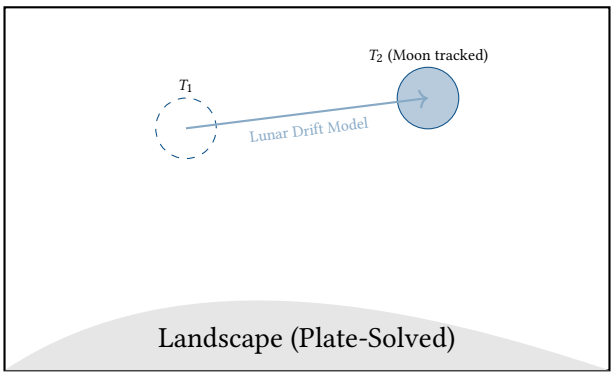


Figure 46. The Eclipse Composite Engine. By plate-solving the wide-field landscape shot to establish a baseline WCS, the system can mathematically superimpose high-resolution, tracked shots of the Moon at their exact predicted celestial coordinates for a given timestamp.

The engine calculates the topocentric lunar drift rate (`lunar_tracking_rate`) and uses the landscape's plate-solved WCS to align the tracked lunar data, creating mathematically accurate composites that bypass the limitations of single-axis tracking on alt-az mounts.

22 Current Status & Future Work

22.1 Milestone History

Table 1. Development milestones.

Version	Name	Date	Phases
v0.1	Celestial Math	February 2026	4
v0.2	Imaging Intelligence	March 2026	6
v0.3	Advanced Astro	March 2026	12
v0.4	Edge Optimization	March 2026	14

The codebase has reached 54,130 lines of Rust with over 2,100 unit tests, representing a 280% increase in code volume since v0.3. The v0.4 milestone successfully delivered the native plate solver, mobile bridge, and power-aware resiliency logic.

22.2 Future Work

- **Planetary derotation.** Integrating WinJUPOS-style gnomonic derotation for fast-rotating planets like Jupiter.
- **Optical mastery.** Automatic aberration correction based on SIP-derived field curvature models.
- **Mobile Bridge expansion.** Extending the FFI to support real-time raw frame streaming over WebSockets for low-latency monitoring on tablets.

23 Conclusion

Open Astro Core v0.4 has evolved from a foundational math library into a comprehensive, production-ready SDK for computational astrophotography. By maintaining a strict pure-function architecture and a high-fidelity digital twin for verification, the project delivers scientific-grade algorithms that are equally at home on a server-grade workstation or a battery-powered mobile device.

The transition to v0.4 marked a shift from purely algorithmic development to architectural robustness, ensuring that the computational primitives of the cosmos are accessible, safe, and deterministic for the next generation of autonomous observatories.

References

[1] J. Meeus, *Astronomical Algorithms*. Willmann-Bell, 1991.

[2] IAU SOFA Board, *Standards of fundamental astronomy*, 2023. [Online]. Available: https://www.iausofa.org/

[3] S. B. Howell, *Handbook of CCD Astronomy*, 2nd. Cambridge University Press, 2006.
[4] J. R. Janesick, *Scientific Charge-Coupled Devices*. SPIE Press, 2001.
[5] E. J. Groth, "A pattern-matching algorithm for two-dimensional coordinate lists," *Astronomical Journal*, vol. 91, pp. 1244–1248, 1986.
[6] D. Lang, D. W. Hogg, K. Mierle, M. Blanton, and S. Roweis, "Astrometry.net: Blind astrometric calibration of arbitrary astronomical images," *Astronomical Journal*, vol. 139, pp. 1782–1800, 2010.
[7] M. A. Fischler and R. C. Bolles, "Random sample consensus," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
[8] G. G. Bennett, "The calculation of astronomical refraction in marine navigation," *Journal of Navigation*, vol. 35, pp. 255–259, 1982.
[9] K. A. Pickering, "The southern limits of the ancient star catalogs," *DIO*, vol. 12, pp. 20–39, 2002.
[10] F. Kasten and A. T. Young, "Revised optical air mass tables and approximation formula," *Applied Optics*, vol. 28, no. 22, pp. 4735–4738, 1989.
[11] E. S. King, *A Manual of Celestial Photography*. Eastern Science Supply Co., 1931.
[12] M. Trueblood and R. Genet, *Microcomputer Control of Telescopes*. Willmann-Bell, 1999.
[13] H. S. Malvar, L.-w. He, and D. Florencio, "High-quality linear interpolation for demosaicing of bayer-patterned color images," in *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, vol. 3, 2004, pp. 485–488.
[14] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd. Cambridge University Press, 2007.
[15] D. Marr and E. Hildreth, "Theory of edge detection," *Proceedings of the Royal Society of London. Series B. Biological Sciences*, vol. 207, no. 1167, pp. 187–217, 1980.
[16] J. W. Tukey, "The future of data analysis," *The Annals of Mathematical Statistics*, vol. 33, no. 1, pp. 1–67, 1962.
[17] R. Glover, "Optimal sub-exposure times for cmos cameras," *Journal of the British Astronomical Association*, vol. 130, no. 4, pp. 212–216, 2020.
[18] A. S. Fruchterman and R. N. Hook, "Drizzle: A method for the linear reconstruction of undersampled images," *Publications of the Astronomical Society of the Pacific*, vol. 114, no. 792, pp. 144–152, 2002.
[19] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 2nd. Prentice Hall, 2002.
[20] M. R. Calabretta and E. W. Greisen, "Representations of celestial coordinates in FITS," *Astronomy & Astrophysics*, vol. 395, pp. 1077–1122, 2002.
[21] D. L. Shupe et al., "The SIP convention for representing distortion in FITS image headers," in *ASP Conference Series*, vol. 347, 2005.
[22] R. E. Strom and S. Yemini, "Typestate: A programming language concept for enhancing software reliability," *IEEE Transactions on Software Engineering*, vol. SE-12, no. 1, pp. 157–171, 1986.
[23] J. D. Giorgini et al., "Jpl's on-line solar system data service," *Bulletin of the American Astronomical Society*, vol. 28, p. 1158, 1996.
[24] Mars Climate Orbiter Mishap Investigation Board, "Phase I report," NASA, Tech. Rep., 1999.
[25] P. J. Huber, *Robust Statistics*. John Wiley & Sons, 1981.
[26] D. J. Schroeder, *Astronomical Optics*, 2nd. Academic Press, 1999.
[27] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
[28] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
[29] P. T. Wallace, "Rigorous pointing models for telescope mounts," *Observatory Operations to Optimize Scientific Return*, vol. 2199, pp. 170–181, 1994.
[30] N. M. Law, C. D. Mackay, and J. E. Baldwin, "Lucky imaging: High angular resolution imaging in the visible from the ground," *Astronomy & Astrophysics*, vol. 446, no. 2, pp. 739–745, 2006.
[31] D. L. Fried, "Optical resolution through a randomly inhomogeneous medium for very long and very short exposures," *Journal of the Optical Society of America*, vol. 56, no. 10, pp. 1372–1379, 1966.
[32] H. Choset, "Coverage for robotics – a survey of recent results," *Annals of Mathematics and Artificial Intelligence*, vol. 31, pp. 113–126, 2001.

[33] R. Isermann, J. Schaffnit, and S. Sinsel, “Hardware-in-the-loop simulation for the design and testing of engine-control systems,” *Control Engineering Practice*, vol. 7, no. 5, pp. 643–653, 1999.

[34] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[35] D. Harel, “Statecharts: A visual formalism for complex systems,” *Science of Computer Programming*, vol. 8, no. 3, pp. 231–274, 1987.